1. **True/False Questions** (20% with 2% each question)

Please indicate whether each of the following statements is true or false.

(a) For a Markov Decision Process (MDP), let $v_\pi(s)$ denote the state value of each state s for an action policy π. For two different action policies, $\pi 1$ and $\pi 2$, we have $v_{\pi 1}(s)=v_{\pi 2}(s)$ for each state s. True or False? Answer: False

(b) For an MDP with an action policy π, the state value $v_\pi(s)$ for each state s is equal to the weighted sum of the state-action values $q(s,a)$ for all actions a weighted by the probability for taking each such action a when in state s. True or False? Answer: True

(c) It is not possible to use the Monte Carlo (MC) method to train a reinforcement-learning agent and apply the trained agent to execute the optimal action policy in the same environment in real-time. True or False? Answer: False

(d) Reinforcement learning using the ϵ-greedy policy can converge to the optimal action policy if the ϵ value is small and fixed. True or False? Answer: False

(e) The SARSA approach essentially corresponds to solving $q(s,a)$'s from the Bellman expectation equations for the underlying MDP. True or False? Answer: True

(f) The temporal difference (TD) method produces results with smaller variance when compared with the Monte Carlo (MC) method. This is because the TD updates consider relatively limited randomness in the environment over two consecutive time steps, while the MC method considers all the randomness throughout the whole episode. True or False? Answer: True

(g) Forward and backward temporal-difference (TD) learning are equivalent in terms of the total amount of updates of the Q-values, but the forward TD cannot be applied to select the optimal actions in real-time. True or False? Answer: True

(h) To approximate and learn the Q-values by using neural networks, the main goal of the stochastic gradient-decent method is to avoid much computation in taking averaging over a lot of experience data. True or False? Answer: True.

(i) The policy-gradient method optimizes a given performance objective function, which generally corresponds to the long-term return in the associated MDP. True or False? Answer: False

(j) After updating the policy parameters θ by gradient ascent for a very large amount of experience data, the policy-gradient method can always converge to the optimal action policy for the chosen policy objective function. True or False? Answer: False

2. Dynamic Programming (25%)

Figure 1 shows a simple Markov Decision Process (MDP) with four states {A, B, C, D} where each state has two available actions and each of the actions leads to transition to a different state with a corresponding reward. The reward is determined solely by the current state and the action taken. For example, in state A, taking one action leads to state B with a reward of 0, while taking the other action leads to state C with a reward of -2. For another example, in state B, one action leads to returning to state B with a reward of -1, while taking another action changes the state from B to D with a reward of 1. The discount factor is 0.8.

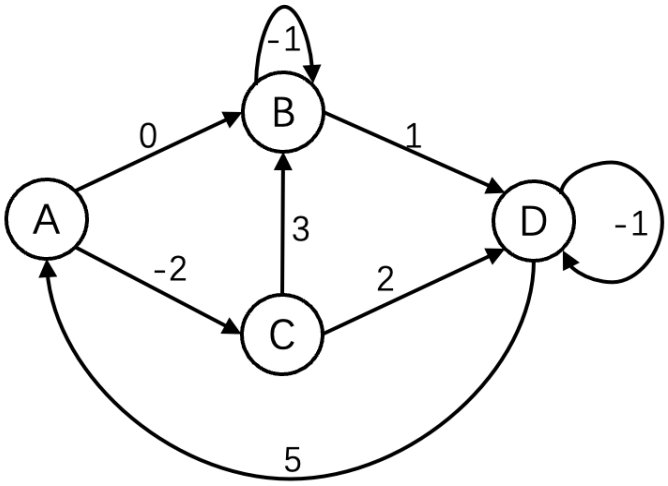


Figure 1. A simple Markov Decision Process (MDP)

(a) Can dynamic programming be applied to find the optimal policy for the above MDP? Briefly explain why. [2]

Answer: Yes, it is so because the full knowledge of the MDP (S, A, P, R, γ), including the states, the actions, the state-transition probabilities, the reward function, and the discount factor, is known.

(b) Let us obtain the state values by iterations using the Bellman optimality equations. Assume all state values are initialized to 0. Calculate the state values for the first two iterations. [8]

Answer:

$v_1(A) = \max\{0, -2\} = 0$

$v_1(B) = \max\{-1, 1\} = 1$

$v_1(C) = \max\{3, 2\} = 3$

$v_1(D) = \max\{-1, 5\} = 5$

$v_2(A) = \max\{0+0.8*v_1(B), -2+0.8*v_1(C)\} = 0.8$

$v_2(B) = \max\{-1+0.8*v_1(B), 1+0.8*v_1(D)\} = 5$

$v_2(C) = \max\{3+0.8*v_1(B), 2+0.8*v_1(D)\} = 6$

$v_2(D) = \max\{-1+0.8*v_1(D), 5+0.8*v_1(A)\} = 5$

(c) After the second iteration, what action should be taken in state A to maximize the long-term return? [6]

Answer:

By using the state values obtained above after the second iteration,

q2(A, A to B) = 0 + 0.8*v2(B) = 0.8*5 = 4

q2(A, A to C) = -2 + 0.8*v2(C) = -2 + 0.8*5 = 2.4

After the second iteration, when in state A, we select the action with the higher state-action value. That is, from the values given above, we choose the action that leads to transition from state A to state B.

(d) How can we determine whether the iterations have produced the optimal state values? [2]

Answer:

The optimal state values are reached when the state values produced by the iterations converge or if all the obtained state values satisfy the Bellman optimality equations.

(e) Consider that a general MDP, not restricted to the one in Figure 1, has an action policy π. That is, the probability of taking an action a while in state s is $\pi(a|s)$. Assume that the state-action values $q_\pi(s, a)$ for all s and a have been obtained. We form a new policy $\hat{\pi}$ by taking a greedy action $\hat{a}(s) = \arg\max_{a \in A} q_\pi(s, a)$ for each state s. Prove that the respective state values $v_\pi(s)$ and $v_{\hat{\pi}}(s)$ for the policies π and $\hat{\pi}$ satisfy $v_\pi(s) \leq v_{\hat{\pi}}(s)$ for all states s. That is, the new policy $\hat{\pi}$ improves the state values over the policy π. [7]

Answer:

$$v_\pi(s) = \sum_a \pi(a|s) q_\pi(s, a) \leq q_\pi(s, \hat{a}(s))$$

$$= E_\pi[R_{t+1} + \gamma q_\pi(S_{t+1}, A_{t+1}) | S_t = s, A_t = \hat{a}(s)]$$

$$= E_{\hat{\pi}}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1})|S_t = s]$$
$$\leq E_{\hat{\pi}}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1} = \hat{a}(S_{t+1}))|S_t = s]$$
$$= E_{\hat{\pi}}[R_{t+1} + \gamma q_{\hat{\pi}}(S_{t+1}, A_{t+1} = \hat{a}(S_{t+1}))|S_t = s]$$
$$= E_{\hat{\pi}}[R_{t+1} + \gamma v_{\hat{\pi}}(S_{t+1})|S_t = s]$$
$$= v_{\hat{\pi}}(s)$$

3. Model-free Algorithms (25%)

Figure 2 shows a 2×3 grid world, which has five states {A, B, C, D, E} and a terminal state T. The available actions include up, down, left, and right, but movement is restricted within the grid. For example, in state A, there are two possible actions: moving down and moving right. In state B, there are three possible actions: moving left, moving right, and moving down. The reward for reaching the terminal state T is 10, while the reward for all other transitions is -1.

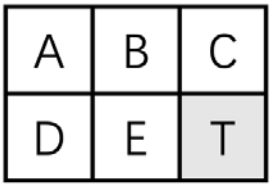


Figure 2. A grid world.

We have collected the following four complete or incomplete episodes using the action policy π:

Episode 1: A, right, -1, B, right, -1, C, down, 10, T

Episode 2: A, down, -1, D, up, -1, A, right

Episode 3: B, right, -1, C, down, 10, T

Episode 4: D, right, -1, E, right, 10, T

(a) Describe the advantages of Temporal Difference (TD) and Monte Carlo (MC) methods as model-free reinforcement-learning techniques. Which method can be applied to an episode that has not reached a terminal state? Which method can be used in non-Markovian environments? [5]

Answer:

TD advantages: Usually more efficient, low-variance results, learn from incomplete episodes, applicable to environments without terminating state, learn online.

MC advantages: Not very sensitive to initial values, zero bias, applicable to non-Markovian environments.

The TD method can use incomplete episodes because it updates values based on bootstrapping, without waiting until the end of an episode.

The MC method can be used in non-Markovian environments because it does not exploit the Markov property and instead learns from complete episode returns.

(b) By using each of the above episodes once where applicable in the MC method, calculate the value of each state with the discount factor $\gamma = 1$ and the learning rate $\alpha = 1$. [4]

Answer:

MC: We can use only the complete episodes 1, 3, and 4.

v(A) = (-1) + (-1) + 10= 8

v(B) = { [(-1) + 10] + [(-1) +10] } / 2 = 9

v(C) = { 10 + 10 } / 2 = 10

v(D) = (-1) + 10 = 9

v(E) = 10

(c) With zero initial state values, $\gamma = 1$ and $\alpha = 1$, use TD learning to update the values of all states according to the order of the experience data in Episode 1, 2, 3 and then 4. Show your result after each update is performed. [4]

Answer:

According to the specified update order, the TD updates are:

v(A) = -1 + γ v(B) = -1 + 0 = -1

v(B) = -1 + γ v(C) = -1 + 0 = -1

v(C) = 10 + γ v(T) = -1 + 0 = 10

v(A) = -1 + γ v(D) = -1 + 0 = -1

v(D) = -1 + γ v(A) = -1 + (-1) = -2

v(B) = -1 + γ v(C) = -1 + 10 = 9

v(C) = 10 + γ v(T) = 10

v(D) = -1 + γ v(E) = -1 + 0 = -1

v(E) = 10 + γ v(T) = 10 + 0 = 10

(c) All Q-values are initialized to zero. Assume the SARSA algorithm is used to update the Q-values with the discount factor $\gamma = 1$ and the learning rate $\alpha = 1$. Fill in the table below after

all Q-value updates by using all (S, A, R, S', A') data once first in Episode 1, 2, 3 and then 4. n/a stands for "not applicable" in the table. [8]

Q(s, a)	A	B	C	D	E
left	n/a			n/a	
right			n/a		
down				n/a	n/a
up	n/a	n/a	n/a		

Answer:

Q(s, a)	A	B	C	D	E
left	n/a	0	0	n/a	0
right	-1	9	n/a	-1	10
down	-1	0	10	n/a	n/a
up	n/a	n/a	n/a	-2	0

(d) Assume that many episodes like the ones given above are available. Suggest a learning algorithm that can use the available data to update and obtain the optimal Q-values and thus the optimal policy π^*. Briefly explain your result. [4]

Answer:

One can process the episode data and update the Q-values according to the Q-learning algorithm. Since Q-learning takes actions that maximize the Q-values, the algorithm will effectively solve the Bellman optimality equations and thus produce the optimal Q-values and policy π^*.

4. RL for Base-Station Selections for Downlink Transmissions to Users (30%)

Consider a cellular network with M base stations, indexed by $m = 1$ to M, and N users, indexed by $n = 1$ to N. The service area consists of 10x10 square locations, indexed by $k = 1$ to 100. The following figure shows a network with M=3 base stations (BS) installed at fixed locations and $N = 5$ mobile users, denoted as User Equipment (UE). Multiple users and a base station can be located at the same location. Assume that the system is discrete time. Let the location of user n at the beginning of time slot t be denoted by $L_n(t)$ =1 to 100. At the end of each time slot t, each user randomly moves to one of his/her neigboring locations with equal probability without leaving the service area.

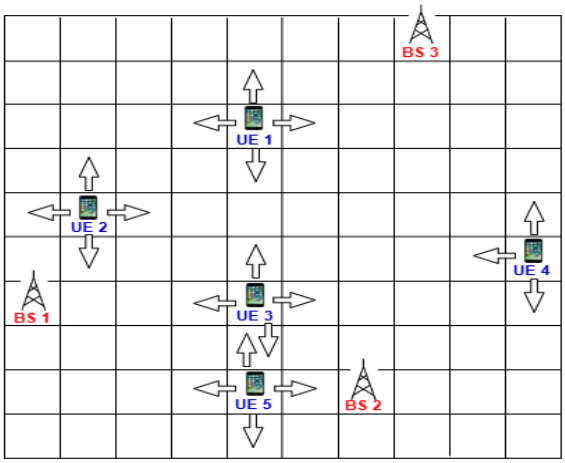


Figure 3. A cellular network with multiple base stations and users.

Assume that all base stations always have data to transmit downlink towards each user. The network is required to select *one and only one* of the M base stations to transmit data to each of the N users in every time slot. As a result, each user receives transmission from one and only one base station in each time slot, although a base station can be selected to transmit to more than one user at a time. Let $r_{mn}(t)$ be the data rate for transmission from base station m to user n if base station m transmits *only* to user n in time slot t. It is assumed that given the base stations at fixed locations, the data rates $r_{mn}(t)$ for all combinations of base stations m and user n can be uniquely determined based on the location $L_n(t)$ of each user n at time t. If base station m is selected to transmit to $C_m(t)$ users simultaneously at time t, the data rate for each such user n being served become $\frac{1}{C_m(t)} r_{mn}(t)$ where $C_m(t)$ can be 1 to N depending on the base-station selections. The idea of the data-rate reduction by the factor of $C_m(t)$ is to account for the sharing of resources such as splitting the transmission power at the base station for simultaneous transmissions to the $C_m(t)$ users. The total data rate of the network in time slot t is the sum of all data rates for all N users in time slot t.

An example of base-station selections is given in Figure 4. For example, BS 1 transmits to UE3, BS 2 serves UE2, UE4 and UE5, and BS 3 transmits to UE1 in time slot 3. Correspondingly, $C_1(3) = 1$, $C_2(3) = 3$, and $C_3(3) = 1$. For the users served by BS 2 in time slot 3, the data rates for UE2, UE4 and UE5 are given by $\frac{1}{C_2(3)} r_{2n}(3)$ where $r_{2n}(3)$ denotes the data rate for user n if BS 2 serves only user n in time slot 3 with n being 2, 4 and 5, respectively. The total data rate of all users in the network in time slot 3 is equal to $\frac{1}{C_1(3)} r_{13}(3) + \frac{1}{C_2(3)} r_{22}(3) + \frac{1}{C_2(3)} r_{24}(3) + \frac{1}{C_2(3)} r_{25}(3) + \frac{1}{C_3(3)} r_{31}(3)$.

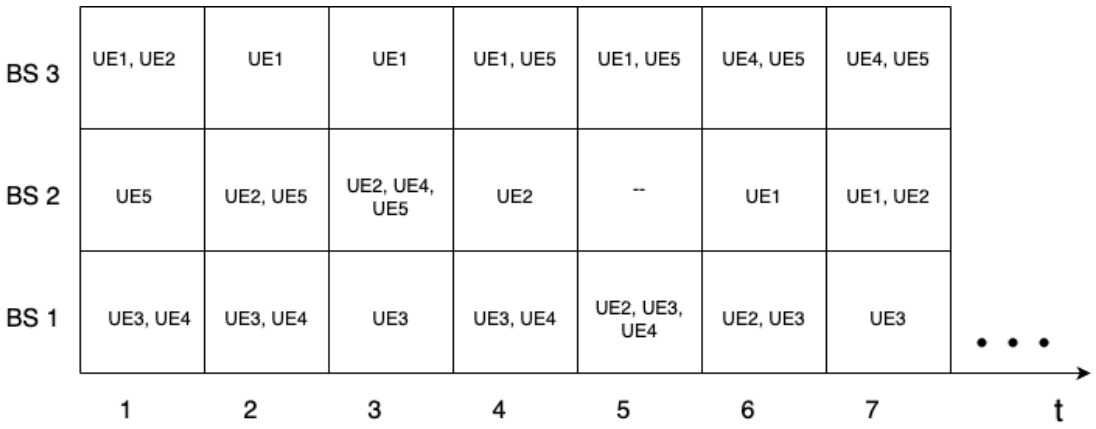


Figure 4. An example of base-station selections for transmission to users.

A reinforcement-learning agent is designed to optimally select one of the M base stations to transmit data downlink to each of the N users in every time slot such that the long-term total data rate with a reward discount factor γ is maximized. At the beginning of each time slot t, the agent is assumed to know the location $L_n(t)$ of every user n and is thus able to determine the data rates $r_{mn}(t)$ for all combinations of base station m and user n. The action, which selects the base stations for transmission to the users, is made for transmissions during every time slot t. The associated immediate reward R_{t+1} is assumed to be known to the agent at the beginning of the following time slot $t+1$.

Let S_t and A_t be the system state and action taken at the beginning of time slot t, respectively, and R_{t+1} denote the immediate reward associated with S_t and A_t and observed by the agent at the beginning of time slot $t+1$.

(a) Define the system state S_t at the beginning of time slot t and briefly explain your result. [5]

Answer: The system state $S_t = \{L_n(t),\ n = 1\ to\ N\}$ where $L_n(t)$ is the location (index) of user n at time t. This state definition is appropriate because the user locations form a Markov process and the user locations in the next time slot depend solely on their locations in the current time slot.

(b) Define the action $A(t)$ in time slot t and briefly explain if there is any constraint on the action. [Hint: Consider an indicator function $\delta_{mn}(t)$=1 or 0 to indicate whether base station m is selected to transmit to user n in time slot t.] [5]

Answer: The action $A(t) = \{\delta_{mn}(t), m = 1\ to\ M, n = 1\ to\ N\}$ where $\delta_{mn}(t)$=1 if base station m is chosen to transmit to user n in time slot t. Otherwise, $\delta_{mn}(t)$=0. To ensure each

user is served by one and only one base station, we require $\sum_{m=1}^{M} \delta_{mn}(t) = 1$ for every user n at every time t.

(c) Using the notation above, what is the immediate reward R_{t+1} for the system state S_t and action A_t taken by the agent in time slot t? [5]

Answer: The immediate reward R_{t+1} is equal to the total data rate for all users served by all base stations in time slot t:

$$R_{t+1} = \sum_{m=1}^{M} \frac{1}{C_m(t)} \sum_{n=1}^{N} r_{mn}(t)\delta_{mn}(t)$$

where $C_m(t) = \sum_{n=1}^{N} \delta_{mn}(t)$ and $\delta_{mn}(t)$ is 1 if base station m is selected to transmit to user n at time t. Otherwise, $\delta_{mn}(t) = 0$.

(d) Briefly describe how the system state change from S_t to S_{t+1}? How does the action A_t taken by the agent at time t affect the state change? [4]

Answer: Since the system state S_t only reflects the user locations, the system state changes from S_t to S_{t+1} according to the movement to one of the neighboring locations of each user with equal probability. In this case, the action A_t taken by the agent does not affect the state change.

(e) Describe briefly the factors and tradeoffs considered by the agent in the base-station selections. [4]

Answer: Intuitively, given the current user locations, the agent may choose a base station m that can transmit at a high data rate $r_{mn}(t)$ for user n. Doing so would improve the total data rate for all users. On the other hand, as users move around randomly, it is possible that many users come close to a particular base station. Selecting the base station to transmit to multiple users reduces the data rate for each of the users being served, as described in the question. The agent is to find a balanced approach to maximizing the total data rates for all users.

(f) If the Q-learning method is used to train the agent and apply the chosen actions to the network in real-time, provide the steps involved in the method and identify how the optimal base-station selections can be obtained. [7]

Answer: Since the system does not have any terminating state, each episode is assumed to contain T time slots long. The Q-learning method consists of the following steps.

i) Initialize episode index $k \leftarrow 1$, time $t \leftarrow 0$, the state-action function $Q(S, A) \leftarrow 1$ for all state S and action A, and the largest relative error $E_{max} \leftarrow 0$.

ii) Choose the $\in$-greedy policy with $\epsilon \leftarrow 1$ as the action policy π for the base-station selections based on the current state-action $Q(S, A)$ values.

iii) Set $t \leftarrow t + 1$.

iv) Observe the current system state S and take the action A using the policy π.

v) Observe the immediate reward (i.e., the total data rate) R and the next state S' in the next time slot.

vi) Using the current state S, action A, immediate reward R and the next state S', update the Q-value as follows:

Find TD error: $\Delta \leftarrow R + \gamma \, max_a Q(S', \mathrm{a}) - Q(S, A)$

If $|\frac{\Delta}{Q(S, A)}| > \mathrm{E}_{max}$, set $\mathrm{E}_{max} \leftarrow |\frac{\Delta}{Q(S, A)}|$ (*)

$Q(S, A) \leftarrow Q(S, A) + \alpha\Delta$

v) If $t = T$ (i.e., the end time for each episode), set $k \leftarrow k + 1$ and $\epsilon = \frac{1}{k}$ to form the new ϵ-greedy policy π.

vi) If the largest relative target error E_{max} from (*) is less than a small threshold such as 10^{-3}, $Q(S, A)$ for all S and A have converged and the algorithm thus stops. Otherwise, set $t = 0$ and $\mathrm{E}_{max} = 0$, and repeat from Step iii).

When the method converges, the optimal policy is obtained by

$$\pi^*(a|s) = \lim_{k \to \infty} 1(a = \arg\max_{a' \in A} Q_k(s, a')) \quad \text{for every state } s.$$